

 Order Within –01Hrs 28Mins 15Secs for Monday despatch



Search for products...

 SEARCH



ACCOUNT  SAVED



 SHOP ALL PRODUCTS ▾

BLOG ▾

SUPPORT

CONTACT US

 **BUY A GIFT CARD**

TRACK MY ORDER

I2S AUDIO ON THE ESP32-S3

 Posted: March 23, 2026  Last Updated: June 8, 2026

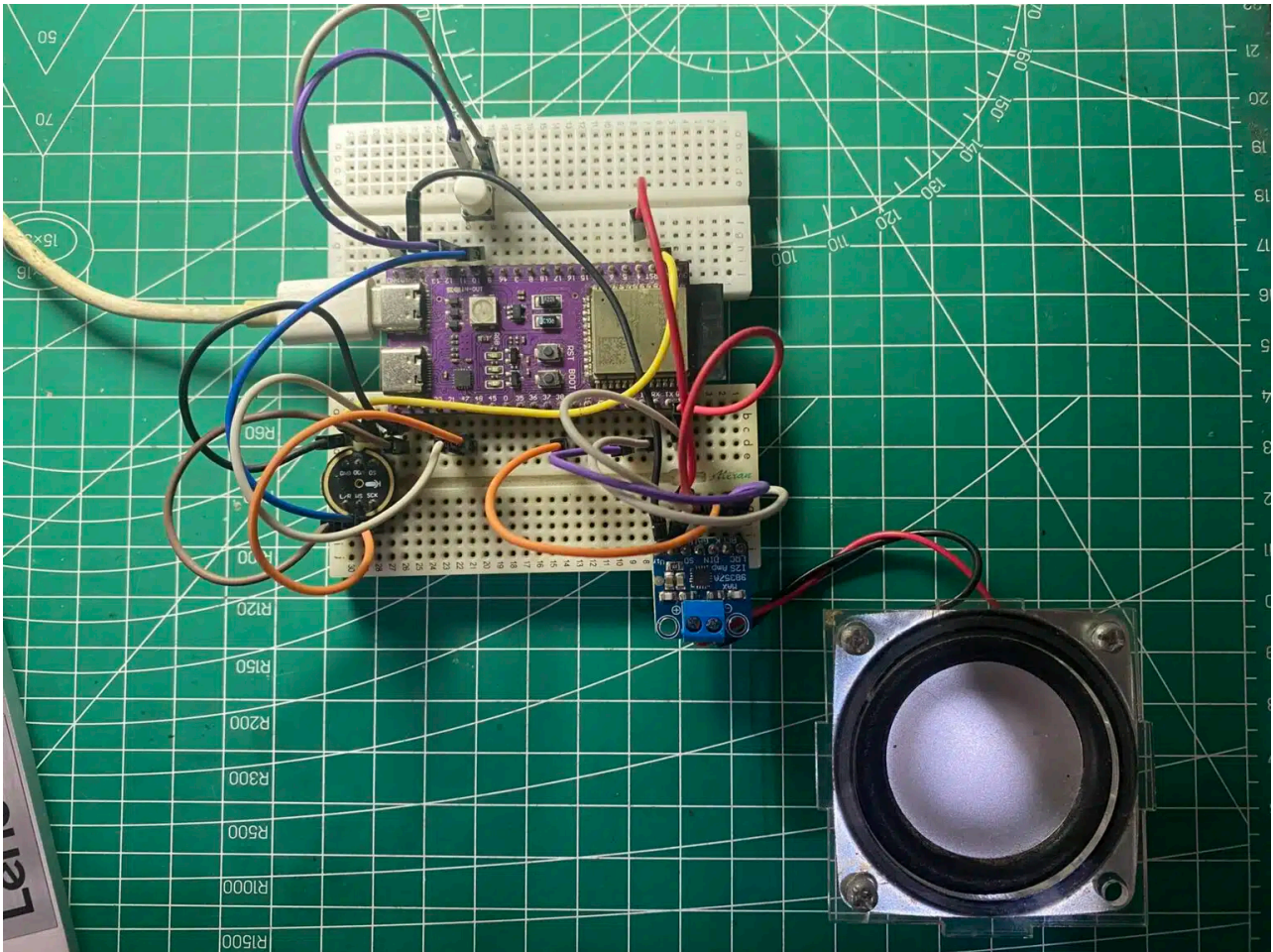
CONTENTS



1. INTRODUCTION
2. SAMPLING CHARACTERISTICS OF I2S
3. SAMPLE CODE FOR THE MAX98357A
4. SAMPLE CODE FOR THE INMP441
5. WHAT'S NEXT?



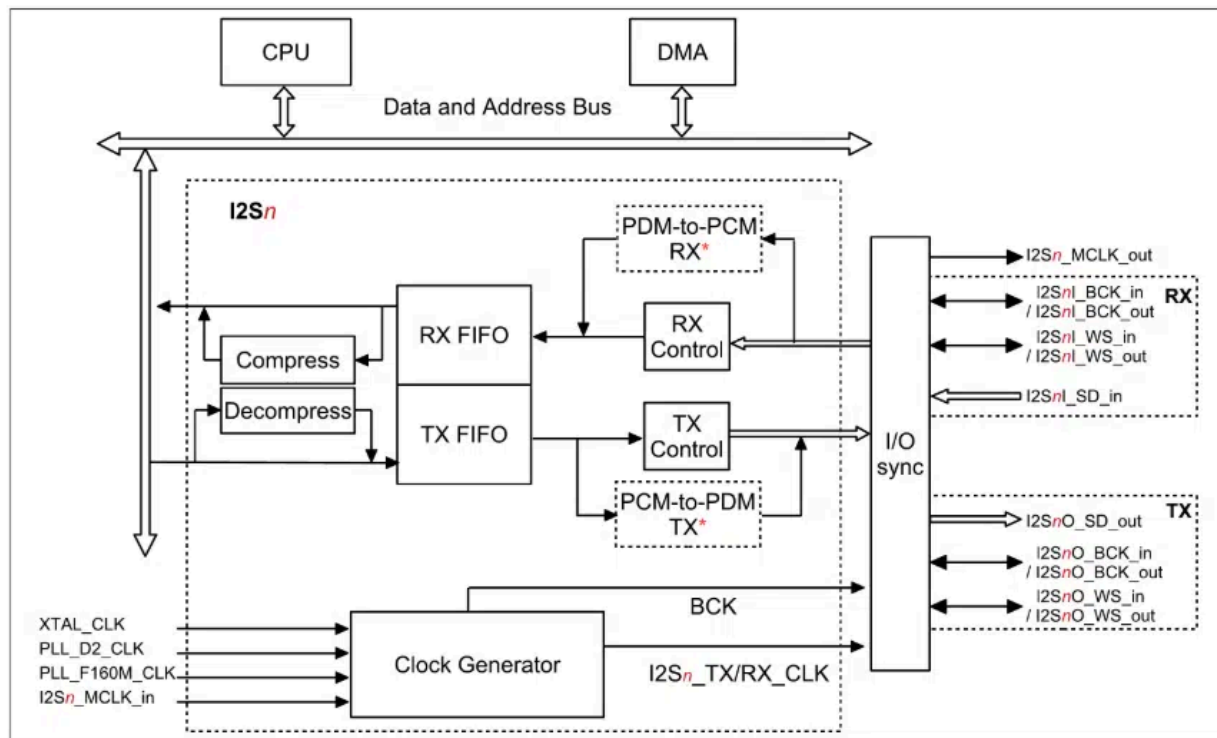
Wanting to implement digital audio on your ESP32-S3? Learn more about running I2S on an ESP32S3 dev board through this [ESP32 I2S audio tutorial](#).



INTRODUCTION

Last time, you learned about the basics of I2S through [Introduction to I2S Audio](#). This time, you can implement the full I2S protocol on the ESP32-S3. The ESP32-S3 has two hardware I2Ss, namely I2S0 and I2S1, which can stream digital media and are especially suited for multimedia audio applications. Below is the hardware block diagram of the ESP32-S3 I2S.





Note: PDM-to-PCM RX and PCM-to-PDM TX are only supported by I2S0.

Figure 28.4-1. ESP32-S3 I2S System Diagram

As was discussed in the [Introduction to I2S Audio](#), there are three main lines for I2S. These are:

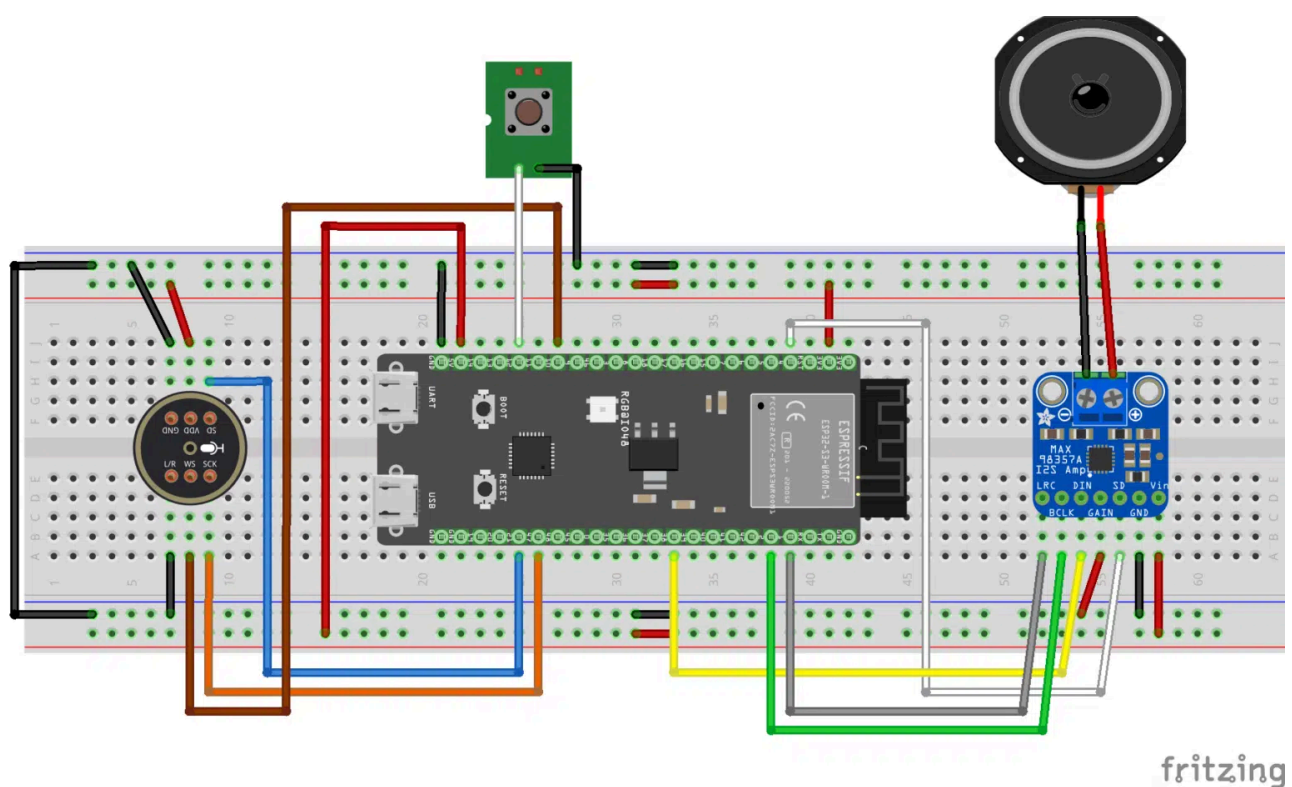
- BCK – Bit clock Signal (or SCK)
- WS – Channel/Word Select Signal
- SD – Serial Data signal (Input or Output)

So, you have to connect these necessary I2S lines of your ESP32-S3 accordingly. As examples, here is an I2S microphone ([INMP441](#)) and a digital audio amplifier ([MAX98357A](#)) connected to an ESP32-S3.

INMP441 PIN	ESP32-S3 PIN	FUNCTION
VCC	3.3V	Power supply (3.3V only)
GND	GND	Ground
SCK (BCLK)	GPIO 47	I2S Bit Clock (BCLK)
WS (LRCLK)	GPIO 10	I2S Word Select (LRCLK)
SD (DOUT)	GPIO 21	I2S Serial Data Input (from mic)
L/R	GND	Left Channel Mic Select

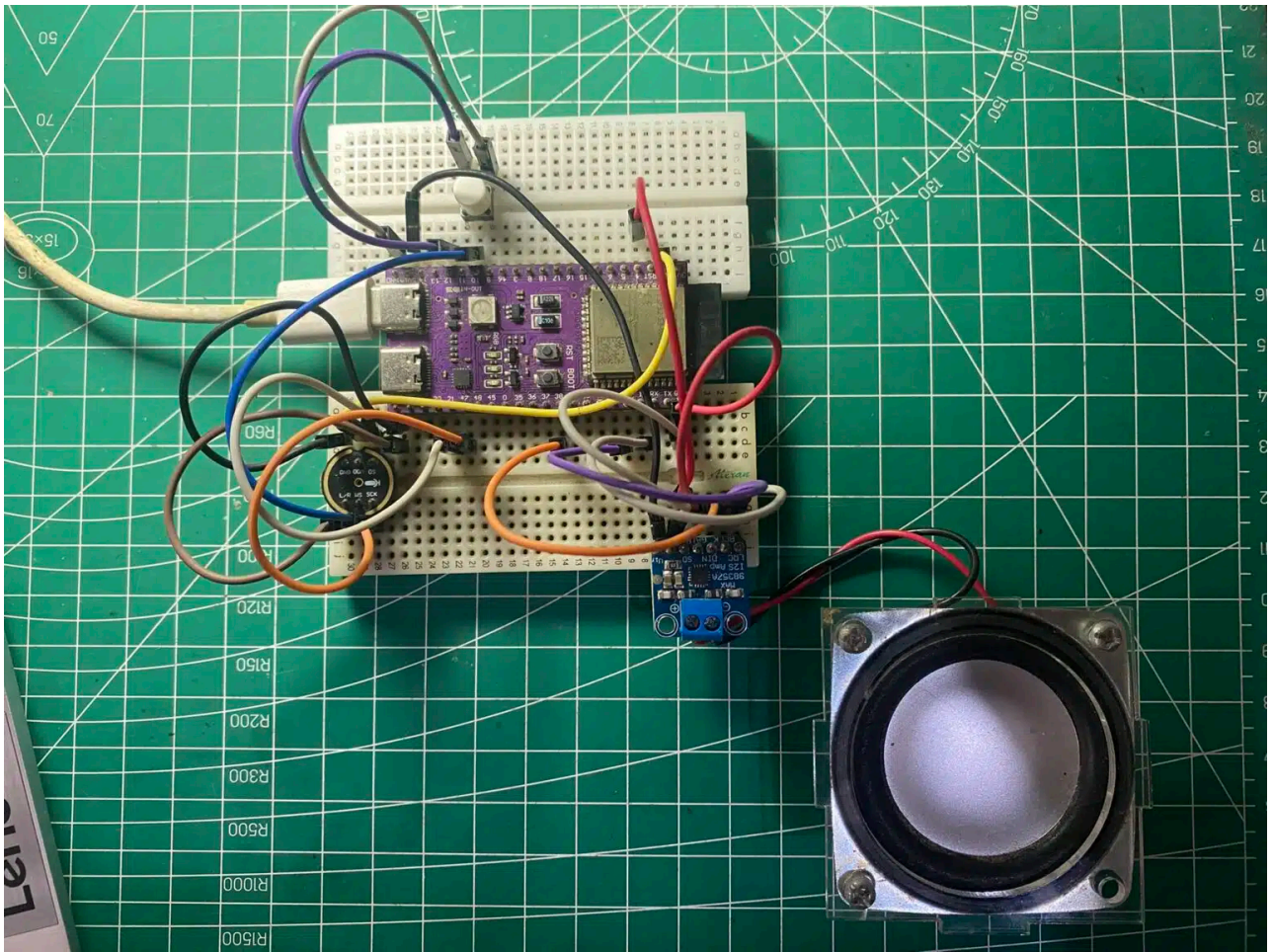
MAX98357A PIN	ESP32-S3 PIN	FUNCTION
VIN	3.3V or 5V	Power supply (supports both)
GND	GND	Ground
BCLK	GPIO 2	I2S Bit Clock
LRCLK	GPIO 1	I2S Word Select (Left/Right Clock)
DIN	GPIO 38	I2S Data Output from ESP32
GAIN	GND	Sets output gain 12 dB
SD	GPIO 4	Optional shutdown control

Here, the Gain pin is set to GND, indicating a gain of 12dB. The SD (Shutdown) pin can be assigned to any free GPIO port, so you can disable the speaker output.



An optional push button is added for future use.





SAMPLING CHARACTERISTICS OF I2S



I2S samples audio data. With this, you should be ready with details about your preferred Sample Rate, Bit Resolution, and Number of Audio Channels involved. These parameters will help you determine the actual I2S clock speed (SCK/BCK). The formula for determining the I2S clock is:

$$\text{BCK} = \text{Sampling Rate} * \text{Bit resolution} * \text{Number of Channels}$$

So, for example, if you want to record or transmit CD-quality audio, you may need parameters similar below:

Sampling Rate = 44100 Hz

Bit Resolution = 16 bits

Channels = x2 (Stereo)

To get a BCK of about 1.4 MHz

$$\text{BCK} = 44100\text{Hz} * 16 \text{ Bits} * 2 \text{ Channels} = 1411200 \text{ Hz}$$

Now that you've learned the signals and parameters to use for I2S, you're ready to code firmware.

SAMPLE CODE FOR THE MAX98357A

Below is a sample code that generates a tone based on a waveform on the MAX98357A digital audio amp. The waveform generated is a triangular wave.

```
1  #include <driver/i2s.h>
2
3  #define I2S_SPK_BCLK 2
4  #define I2S_SPK_LRC 1
5  #define I2S_SPK_DOUT 38
6
7  #define I2S_SD_SPK 4
8  #define I2S_SPKR_PORT I2S_NUM_0
9
10 #define BUTTON 12
11
12 const int sampleRate = 44100;    // Hz
13 const int frequency = 1000;      // Hz
14 const int amplitude = 1000;      // -amplitude +amplitude
15 const int samplesPerCycle = sampleRate / frequency;
16
17 int16_t waveBuffer[samplesPerCycle]; // store 1 cycle of triangular wave
18
19
```

```

20 void generateTriangularWave() {
21     for (int i = 0; i < samplesPerCycle; ++i) {
22         float phase = (float)i / samplesPerCycle;
23         float value = (phase < 0.5) ? (2 * phase) : (2 * (1 - phase));
24         waveBuffer[i] = (int16_t)((value * 2 - 1) * amplitude); // Scale to [-am
25     }
26 }
27
28 void setupI2S() {

```

The triangle wave is based on the generateTriangularWave() function. This function generates one full cycle of a triangular wave with a 16-bit resolution.

```

1 void generateTriangularWave() {
2     for (int i = 0; i < samplesPerCycle; ++i) {
3         float phase = (float)i / samplesPerCycle;
4         float value = (phase < 0.5) ? (2 * phase) : (2 * (1 - phase));
5         waveBuffer[i] = (int16_t)((value * 2 - 1) * amplitude); // Scale to [-ampl
6     }
7 }

```

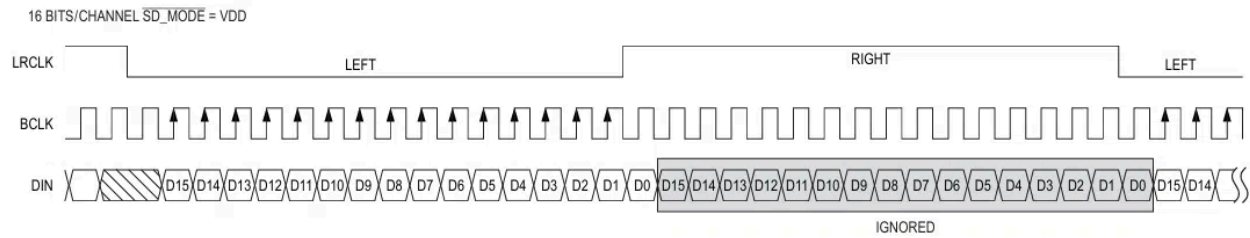
The include file is the <driver/i2s.h>, which is an ESP-IDF type header. The definitions of the pins are also included, together with the waveform sample parameters.

```

1 #include <driver/i2s.h>
2
3 #define I2S_SPK_BCLK 2
4 #define I2S_SPK_LRC 1
5 #define I2S_SPK_DOUT 38
6
7 #define I2S_SD_SPK 4
8 #define I2S_SPKR_PORT I2S_NUM_0
9
10 #define BUTTON 12
11
12 const int sampleRate = 44100; // Hz
13 const int frequency = 1000; // Hz
14 const int amplitude = 1000; // -amplitude +amplitude
15 const int samplesPerCycle = sampleRate / frequency;
16
17 int16_t waveBuffer[samplesPerCycle]; // store 1 cycle of triangular wave

```

Note, here, a 16-bit signed integer array variable (waveBuffer[]) is used to hold the audio sample data. The expected waveform output to the MAX98357A should look similar to below. Only the Left Channel is used.



Upon setup, you'll see the necessary configuration parameters to set up I2S with your DAC. This includes both the I2S config and pin setup. The bits per sample are set to 16 bits using only the left channel, as expected for the DAC. Note that the configuration uses DMA, which helps preserve precious CPU cycles when streaming I2S.

```

1 void setupI2S() {
2     const i2s_config_t i2s_config = {
3         .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_TX),
4         .sample_rate = sampleRate,
5         .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT,
6         .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
7         .communication_format = I2S_COMM_FORMAT_I2S_MSB,
8         .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
9         .dma_buf_count = 8,
10        .dma_buf_len = 32,
11        .use_apll = false,
12        .tx_desc_auto_clear = true,
13        .fixed_mclk = 0
14    };
15
16    const i2s_pin_config_t pin_config = {
17        .bck_io_num = I2S_SPK_BCLK,
18        .ws_io_num = I2S_SPK_LRC,
19        .data_out_num = I2S_SPK_DOUT,
20        .data_in_num = I2S_PIN_NO_CHANGE
21    };
22
23    i2s_driver_install(I2S_SPKR_PORT, &i2s_config, 0, NULL);
24    i2s_set_pin(I2S_SPKR_PORT, &pin_config);
25 }

```

A button press is detected in a loop to know when to proceed to write audio data to the I2S.

i2s_write() is issued with the necessary parameters, including those of your audio data buffer. The write will wait until the specified number of bytes are successfully written to the I2S DMA buffer. Once it's written, the DMA will do its job of providing data to I2S irrespective of your other commands to the MCU. The write is repeated in a loop, generating a seamless audio tone, at the same time, acquiring I2S data. Correspondingly, the shutdown pin of the DAC is activated when there is no I2S write.

```
1  uint8_t toggle = 0;
2
3  void loop() {
4
5      size_t bytesWritten;
6
7      if(digitalRead(BUTTON) == 0){
8          delay(300);    // debounce
9          toggle ^= 1;
10         Serial.println("Button Pressed");
11
12         if(toggle){
13             Serial.println("Audio ON");
14             digitalWrite(I2S_SD_SPK, HIGH); // turn ON speaker
15         }else{
16             Serial.println("Audio OFF");
17             digitalWrite(I2S_SD_SPK, LOW);  // turn OFF speaker
18         }
19     }
20
21     // write I2S data to DMA (1 Traingular Wave cycle)
22     if(toggle){
23         if(i2s_write(I2S_NUM_0, waveBuffer, sizeof(waveBuffer), &bytesWritten, p
24             Serial.print("OK"); Serial.println(bytesWritten);
25     }else{
```

Below is a video demo of the I2S audio using a triangular wave:

0:00 / 0:08

SAMPLE CODE FOR THE INMP441

Below is a sample code to simply know that your INMP411 microphone is running. Audio data is graphically output to the serial plotter in real-time.

```
1  #include <driver/i2s.h>
2
3  #define SAMPLE_BUFFER_SIZE 512
4  #define SAMPLE_RATE 16000
5
6  #define I2S_BCLK_MIC 47
7  #define I2S_LRC_MIC 10
8  #define I2S_DIN_MIC 21
9
10 #define I2S_SD_SPK 4 // DAC shutdown pin
11
12 #define I2S_MIC_PORT I2S_NUM_0
13
14 i2s_config_t i2s_config = {
15     .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
16     .sample_rate = SAMPLE_RATE,
17     .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT,
18     .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
19     .communication_format = I2S_COMM_FORMAT_I2S_MSB,
20     .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
21     .dma_buf_count = 4, // you can expereimnet with this value
22     .dma_buf_len = 1024, // you can expereimnet with this value
23     .use_apll = false,
24     .tx_desc_auto_clear = false,
25     .fixed_mclk = 0};
```

You'll see from the defines some parameters needed to set up I2S correctly, including the I2S MIC pinouts. A define for the DAC speaker is also included so that you can make the DAC silent when the MIC is running.

```
1  #include <driver/i2s.h>
2
3  #define SAMPLE_BUFFER_SIZE 512
4  #define SAMPLE_RATE 16000
5
6  #define I2S_BCLK_MIC 47
7  #define I2S_LRC_MIC 10
8  #define I2S_DIN_MIC 21
9
10 #define I2S_SD_SPK 4 // DAC shutdown pin
11
12 #define I2S_MIC_PORT I2S_NUM_0
```

You configure the I2S port for the MIC through the statements below. Here, you'll see that bits per sample is set at 16 bits and uses the left channel only. The configuration also uses DMA by using a large buffer (4 DMA buffers at 1024 in length). You set up I2S through the `i2s_driver_install()` and `i2s_set_pin()` functions. The `pinMode()` and `digitalWrite()` functions are just meant to quiet down the DAC when you are using the mic.

```
1  i2s_config_t i2s_config = {
2      .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
3      .sample_rate = SAMPLE_RATE,
4      .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT,
5      .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
6      .communication_format = I2S_COMM_FORMAT_I2S_MSB,
7      .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
8      .dma_buf_count = 4, // you can experiment with this value
9      .dma_buf_len = 1024, // you can experiment with this value
10     .use_apll = false,
11     .tx_desc_auto_clear = false,
12     .fixed_mclk = 0};
13
14 i2s_pin_config_t i2s_mic_pins = {
15     .bck_io_num = I2S_BCLK_MIC,
16     .ws_io_num = I2S_LRC_MIC,
17     .data_out_num = I2S_PIN_NO_CHANGE,
18     .data_in_num = I2S_DIN_MIC};
19
20 void setup()
21 {
```

```

22
23 Serial.begin(115200);
24
25 i2s_driver_install(I2S_MIC_PORT, &i2s_config, 0, NULL);
26 i2s_set_pin(I2S_MIC_PORT, &i2s_mic_pins);

```

Inside the main loop, you'll see that the audio data raw samples are set at a 16-bit integer type. I2S data is extracted from the microphone through `i2s_read()` with the necessary data handling parameters. The samples are read in 16-bit format, although the INMP441 really outputs 24-bit per channel (so some fidelity is lost). The INMP441 waveform for the left channel can be seen below. Note that you need to GND the L/R channel port to signal to the MIC that you are intending to use the Left channel.

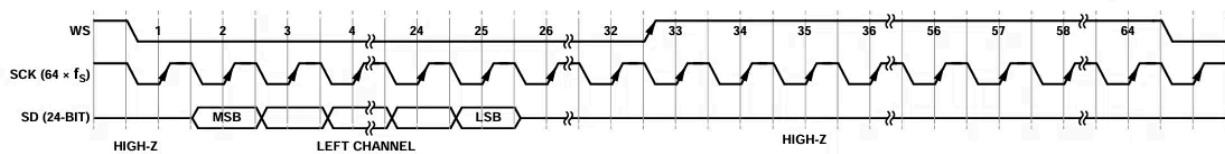


Figure 9. Mono-Output I2S Format Left Channel (L/R = 0)

See the output waveform below. To get an orderly visual, try to whistle into the mic to get a constant frequency waveform. The more you increase the sampling rate, the finer your graph will be.

0:00 / 0:17

You've just learned intermediate ESP32 I2S audio fundamentals. In the next part, you'll integrate the mic and the audio amp to enable [voice recording with an ESP32](#).

WHAT'S NEXT?